MATERIAL DE APOYO

Recursos y siguientes pasos

Referencias, temas para profundizar y por dónde seguir después de la charla.

Preparado para	Fecha y hora	Preparado por
PyCon Panamá 2026	Viernes 23 de octubre de 2026	Ricardo Tovar

01 — Documentación oficial

- Pydantic AI, visión general: https://pydantic.dev/docs/ai/overview/
- Salida estructurada (`output_type`): https://pydantic.dev/docs/ai/core-concepts/output/
- Dependencias y `RunContext`: https://pydantic.dev/docs/ai/core-concepts/dependencies/
- Persistencia del historial: https://pydantic.dev/docs/ai/core-concepts/persistence/
- Pruebas con `TestModel` y `override`: https://pydantic.dev/docs/ai/guides/testing/
- Ejemplo con PostgreSQL y pgvector (RAG): https://pydantic.dev/docs/ai/examples/data-analytics/rag/
- Integración con Logfire: https://pydantic.dev/docs/ai/integrations/logfire/
- Logfire con FastAPI: https://pydantic.dev/docs/logfire/integrations/web-frameworks/fastapi/
- Pydantic, validación: https://pydantic.dev/docs/validation/latest/get-started/
- FastAPI: https://fastapi.tiangolo.com/features/
- `asyncio.timeout` y tareas: https://docs.python.org/3/library/asyncio-task.html
- asyncpg: https://magicstack.github.io/asyncpg/current/

Los ejemplos de este repositorio usan Pydantic AI 2.56.0: https://github.com/pydantic/pydantic-ai/releases/tag/v2.56.0

02 — Referencias de la charla

- Equipo de Pydantic, documentación de Pydantic AI v1.0.0. Fuente de la frase «bring that FastAPI feeling to GenAI app and agent development»: https://raw.githubusercontent.com/pydantic/pydantic-ai/v1.0.0/docs/index.md
- Samuel Colvin, «Pydantic AI v1», 4 de septiembre de 2025: https://pydantic.dev/articles/pydantic-ai-v1
- Justin Schroeder (Standard Agents), «The Future Is Domain-Specific Agents», AI Engineer World's Fair 2026: https://www.youtube.com/watch?v=spNAUEgq_A8
- Vinoth Govindarajan (OpenAI), «Your Agent Didn't Fail. Your Harness Did.», AI Engineer World's Fair 2026: https://www.youtube.com/watch?v=BInpv7lGp1o

03 — Temas para profundizar

- Guardar el historial de conversación en PostgreSQL con una columna `jsonb`.
- Búsqueda vectorial con pgvector y una herramienta de recuperación.
- Evaluaciones: medir la calidad de un agente con casos conocidos antes de cambiarlo.
- Trazas con Logfire para ver cada ejecución, herramienta y validación.
- Reintentos con `ModelRetry` cuando una validación de salida falla.
- Plazos y concurrencia: qué pasa cuando muchas peticiones llegan a la vez.

Multiagentes y grafos quedan fuera de esta charla y de este material. Son lectura posterior.

04 — Siguientes pasos

1. Clona el repositorio y corre `uv run pytest`.
2. Lee `examples/agente.py` y cambia la categoría de salida. Mira qué prueba falla.
3. Escribe una segunda herramienta y pruébala con `TestModel`.
4. Corre el servicio con un modelo y revisa las trazas.
5. Elige un trabajo concreto de tu dominio y define su contexto, sus herramientas y cómo sabrás que salió bien.

05 — Una advertencia

Una salida validada cumple un contrato de forma. No garantiza que el contenido sea verdadero. Prueba y revisa antes de confiar en un agente.